

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	P. Chola Lakshmi Narasimha	Reg. No.:	192325021
Section / Batch:		Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
 - Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: CFG Rule Validation and Parse Tree Construction

START

Define function `parse_cfg(sentence)`

Convert sentence into lowercase words

If number of words is not 5

Return "Invalid Sentence"

Define:

Det = {the, a}

N = {student, teacher, book}

V = {reads, likes}

Check whether:

Word 1 is Det

Word 2 is N

Word 3 is V

Word 4 is Det

Word 5 is N

If all conditions are TRUE

Construct parse tree:

S

├── NP

| | — Det

| | — N

| — VP

| — V

| — NP

| — Det

| — N

Return parse tree

Otherwise

Return "Invalid Sentence"

END FUNCTION

Read sentence from user

Call parse_cfg(sentence)

Display result

STOP

Program

```
def parse_cfg(sentence):
```

```
    words = sentence.lower().split()
```

```
    if len(words) != 5:
```

```
        return "Invalid Sentence"
```

```
    determiners = {"the", "a"}
```

```
nouns = {"student", "teacher", "book"}
```

```
verbs = {"reads", "likes"}
```

```
if (words[0] in determiners and
```

```
    words[1] in nouns and
```

```
    words[2] in verbs and
```

```
    words[3] in determiners and
```

```
    words[4] in nouns):
```

```
    parse_tree = (
```

```
        "S",
```

```
        ("NP",
```

```
            ("Det", words[0]),
```

```
            ("N", words[1])),
```

```
        ("VP",
```

```
            ("V", words[2]),
```

```
            ("NP",
```

```
                ("Det", words[3]),
```

```
                ("N", words[4])))
```

```
    )
```

```
    return parse_tree
```

```
    return "Invalid Sentence"
```

```
# Main program
```

```
sentence = input("Enter a sentence: ")
```

```
result = parse_cfg(sentence)
```

```
if result == "Invalid Sentence":
```

```
    print(result)
```

```
else:
```

```
    print("Valid Parse Tree:")
```

```
    print(result)
```

Output:

Enter a sentence: the student reads a book

Valid Parse Tree:

('S', ('NP', ('Det', 'the'), ('N', 'student'))),

('VP', ('V', 'reads'),

('NP', ('Det', 'a'), ('N', 'book'))))

Experiment 2: Agreement and Feature Structure Checking

Pseudocode:

START

Define function check_agreement(subject, verb)

Define subject features:

he → singular

she → singular

it → singular

they → plural

Define verb features:

runs → singular

writes → singular

run → plural

write → plural

Convert subject and verb to lowercase

If subject or verb is not present

Return False

Get number feature of subject

Get number feature of verb

If subject number equals verb number

Return True

Else

Return False

END FUNCTION

Read subject

Read verb

Call check_agreement(subject, verb)

Display result

STOP

Program:

```
def check_agreement(subject, verb):
```

```
    subject_features = {
```

```
        "he": "singular",
```

```
        "she": "singular",
```

```
        "it": "singular",
```

```
        "they": "plural"
```

```
    }
```

```
    verb_features = {
```

```
        "runs": "singular",
```

```
        "writes": "singular",
```

```
        "run": "plural",
```

```
        "write": "plural"

    }

    subject = subject.lower()

    verb = verb.lower()

    if subject not in subject_features:

        return False

    if verb not in verb_features:

        return False

    if subject_features[subject] == verb_features[verb]:

        return True

    else:

        return False

# Main program

subject = input("Enter subject: ")

verb = input("Enter verb: ")

result = check_agreement(subject, verb)

print(result)
```

Output:

Enter subject: he

Enter verb: runs

True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

START

Define function sentence_probability(sentence)

Define grammar probabilities:

$S \rightarrow NP VP = 1.0$

$NP \rightarrow John = 0.6$

$NP \rightarrow Mary = 0.4$

$VP \rightarrow runs = 0.5$

$VP \rightarrow walks = 0.5$

Split sentence into words

If number of words is not 2

Return 0

First word = subject

Second word = verb

If subject is not present in NP grammar

Return 0

If verb is not present in VP grammar

Return 0

Get:

S probability

NP probability

VP probability

Calculate:

Probability =

S probability $\times$ NP probability $\times$ VP probability

Return probability

END FUNCTION

Read sentence

Call sentence_probability(sentence)

Display probability

STOP

Python Program

```
def sentence_probability(sentence):
```

```
    grammar = {
```

```
        "S": {
```

```
            "NP VP": 1.0
```

```
        },
```

```
        "NP": {
```

```
            "John": 0.6,
```

```
            "Mary": 0.4
```

```
        },
```

```
        "VP": {
```

```
"runs": 0.5,
```

```
"walks": 0.5
```

```
}
```

```
}
```

```
words = sentence.split()
```

```
if len(words) != 2:
```

```
    return 0.0
```

```
subject = words[0]
```

```
verb = words[1]
```

```
if subject not in grammar["NP"]:
```

```
    return 0.0
```

```
if verb not in grammar["VP"]:
```

```
    return 0.0
```

```
s_probability = grammar["S"]["NP VP"]
```

```
np_probability = grammar["NP"][subject]
```

```
vp_probability = grammar["VP"][verb]
```

```
probability = (
```

```
    s_probability *
```

```
    np_probability *
```

```
    vp_probability
```

```
)
```

return probability

```
# Main program
```

```
sentence = input("Enter a two-word sentence: ")
```

```
probability = sentence_probability(sentence)
```

```
print("Sentence Probability:", probability)
```

OUTPUT:

Enter a two-word sentence: John runs

Sentence Probability: 0.3

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning		
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts		
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation		
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools		
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results		
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation		
Q. No. / Criteria Level	Understandin g of Concepts	Experimenta l Design	Use of Tools	Analysis of Results	Documen tation	Sub. Total	Tota l %
1							
2							

3							
Faculty In-charge		Course Coordinator			Program Director		
Signature: _____		Signature: _____			Signature: _____		
Date: _____		Date: _____			Date: _____		